

レポート課題 1

05-191009 奥村真司

問 1

動作例

```
# sum_to 10;;  
- : int = 55  
# gcd 6 15;;  
- : int = 3  
# gcd 39 91;;  
- : int = 13  
# is_prime 1;;  
- : bool = false  
# is_prime 2;;  
- : bool = true  
# is_prime 6;;  
- : bool = false  
# is_prime 1000000007;;  
- : bool = true
```

問 2

動作例

```
# twice (fun x -> x*x) 3;;  
- : int = 81  
# repeat (fun x -> x*x) 3 2;;  
- : int = 256
```

問 3

動作例

```
# sum_to 10;;  
- : int = 55  
# is_prime 57;;  
- : bool = false  
# is_prime 59;;  
- : bool = true  
# is_prime 1000000007;;  
- : bool = true
```

```
# gcd 4826731 2134197;;  
- : int = 377
```

考察

`sum_to` を例にとって説明する。`sum_to` の定義は $\text{sum_to } n = \text{if } n=0 \text{ then } 0 \text{ else } n + f(n-1)$ である。これに関数の引数を加えた $\text{sum_to_sub } f \ n = \text{if } n=0 \text{ then } 0 \text{ else } n + f(n-1)$ を考える (`sum_to_sub` は再帰関数でないことに注意)。再帰関数をこのように変形したものともとの再帰関数に与える引数を `fix` に渡すと求めたい値が求まることを確認する。 $\text{fix sum_to_sub } 3 = \text{sum_to_sub } (\text{fix sum_to_sub}) \ 3 = 3 + (\text{fix sum_to_sub}) \ 2$ 、 $\text{fix sum_to_sub } 2 = \text{sum_to_sub } (\text{fix sum_to_sub}) \ 2 = 2 + (\text{fix sum_to_sub}) \ 1$ 、... というように展開されて $3+2+1+0$ が求まる。`sum_to` 以外もどのようにして実装できる。

問 4

動作例

```
# fold_right (fun x l -> x::l) ! [1;2;3;4;5] [] ;;  
- : int list = [1; 2; 3; 4; 5]  
# fold_left (fun l x -> x::l) [] [1;2;3;4;5] ;;  
- : int list = [5; 4; 3; 2; 1]  
# fold_right (fun x y -> 10*y+x) [1;2;3;4;5] 0 ;;  
- : int = 54321  
# fold_left (fun x y -> 10*x+y) 0 [1;2;3;4;5] ;;  
- : int = 12345
```

考察

`fold_right` の定義より、 $\text{fold_right } f \ [x_1; x_2; \dots x_n] \ e = f \ x_1 \ (\text{fold_right } f \ [x_2; \dots x_n] \ e)$ であるからこれを用いて実装した。同様に `fold_left` の定義より $\text{fold_left } f \ e \ [x_1; x_2; \dots x_n] = \text{fold_left } f \ (f \ e \ x_1) \ [x_2; \dots x_n]$ である。これを用いて実装すると呼び出しが自分自身そのままであるから問題の指示どおり末尾再帰になっている。

問 5

動作例

```
# append [1;2;3;4] [5;6;7];;  
- : int list = [1; 2; 3; 4; 5; 6; 7]  
# filter is_prime [1;2;3;4;5;6;7;8;9;10;11;12;13];;  
(* is_prime defined in problem 1 *)  
- : int list = [2; 3; 5; 7; 11; 13]
```

考察

`append` の第 1,2 引数をそれぞれ `p,q` とすると、`append` の定義より `p=[]` のとき `append p q = q`、`p=x::xs` のとき `append p q = x::(append xs q)` なのでこれを用いて実装した。`filter` の第 1 引数の関数を `f`、第 2 引数のリストを `l` とする。`filter` の定義より、`l=[]` のとき `filter f l = []`、`l=x::xs` のとき `f x = true` ならば `x::(filter f xs)`、`false` なら `filter f xs` なのでこれを用いて実装した。

問 6

動作例

```
# append_via_fold_right [1;2;3;4] [5;6;7];;
- : int list = [1; 2; 3; 4; 5; 6; 7]
# filter_via_fold_right is_prime [1;2;3;4;5;6;7;8;9;10;11;12;13];;
(* is_prime defined in problem 1 *)
- : int list = [2; 3; 5; 7; 11; 13]
# append_via_fold_left_fast [1;2;3;4] [5;6;7];;
- : int list = [1; 2; 3; 4; 5; 6; 7]
# filter_via_fold_left_fast is_prime [1;2;3;4;5;6;7;8;9;10;11;12;13];;
(* is_prime defined in problem 1 *)
- : int list = [2; 3; 5; 7; 11; 13]
```

考察

はじめに `fold_right` を用いた実装について考察する。まず `append` について、`list` は定義より後ろに要素を足すのはリストの長さの線形時間かかってしまうので先頭に要素を追加することを考えると、第二引数の `list` を初期値として第一引数の `list` の要素を最後尾から順に先頭に追加していけばよい。これには `fold_right` の形式が適する。次に `filter` であるが `append` の時と同様の理由で、空 `list` から始めて `list` の最後尾から順に条件を満たせば追加することを繰り返すことにすると、要素と `list` を受け取って要素が条件を満たすならば先頭に要素を追加した `list` を、そうでなければそのままのリストを返す関数を用意すれば `fold_right` の形式に適する。

`fold_left` を用いた実装については、直接的に考えるのは難しそうだったので発展 2 の `fold_right` を `fold_left` で表した結果を用いて置換した。(`****.via_fold_left_fast`) 発展 2 が解ける前はリストの最後尾に要素をリストの長さの線形時間かけて挿入する関数を用いてリスト長の 2 乗で動くのを実装した (`****.via_fold_left_slow`) が、`fast` は線形時間で動作する。

問 5 と比較して、書きやすさは今回の課題をやった際は問 5 のほうが書きやすく感じたが、`fold` に慣れれば問 5 のように漸化式を立てることなく、`right` や `left` と形があつてるか見るだけで書けるものが多いので楽なことが多くなるように感じる。

速度を比較するために `benchmark.ml` で実装した関数を用いて時間を計測した。`append`, `append_via_fold_right`, `append_via_fold_left_fast` のいずれも線形オーダーなので 10^5 程度の長さのリストを 2 つ `append` する際に実行時間に大きな差異はなかったが、`append_via_fold_left_fast` 以外の 2 つは末尾再帰でないので配列サイズを 10^7 程度にするとスタックオーバーフローした。`append_via_fold_left_slow` は前述の通り 2 乗のオーダーなので

10^5 で 1 分以上終了しなかった。

問 7

動作例

```
# perm [1;2;3];;
- : int list list =
[[1; 2; 3]; [1; 3; 2]; [2; 1; 3]; [2; 3; 1]; [3; 1; 2]; [3; 2; 1]]
# perm [2;3;5;7];;
- : int list list =
[[2; 3; 5; 7]; [2; 3; 7; 5]; [2; 5; 3; 7]; [2; 5; 7; 3]; [2; 7; 3; 5];
 [2; 7; 5; 3]; [3; 2; 5; 7]; [3; 2; 7; 5]; [3; 5; 2; 7]; [3; 5; 7; 2];
 [3; 7; 2; 5]; [3; 7; 5; 2]; [5; 2; 3; 7]; [5; 2; 7; 3]; [5; 3; 2; 7];
 [5; 3; 7; 2]; [5; 7; 2; 3]; [5; 7; 3; 2]; [7; 2; 3; 5]; [7; 2; 5; 3];
 [7; 3; 2; 5]; [7; 3; 5; 2]; [7; 5; 2; 3]; [7; 5; 3; 2]]
```

考察

perm の結果のリスト内の各リストを先頭要素ごとに分類し、それらの先頭要素を取り除いたものは perm の引数のリストからその要素を取り除いたリストを perm に渡した結果なので、引数のリストの各要素を選び、それ以外の要素からなるリストに対しての perm の結果の各リストの先頭に選んだ要素を追加し (これを map でやった)、それをすべて @ でつなげたものを返せばよい (つないでいる途中のものをアキュムレータに入れつつ再帰で実装した)

発展 1

動作例

```
# reverse_via_fold_left [1;2;3;4;5];;
- : int list = [5; 4; 3; 2; 1]
# reverse_via_fold_right [1;2;3;4;5];;
- : int list = [5; 4; 3; 2; 1]
```

考察

$\text{reverse } [x_1; x_2; \dots; x_n] = x_n :: (\text{reverse } [x_1; x_2; \dots; x_{n-1}])$ なのでこれは fold_left の形式に適する。fold_right を用いて書く方法は問 6 と同様に発展 2 の結果と fold_left で書いた結果を用いて書いた。

発展 2

動作例

```
# fold_right_via_fold_left (fun x l -> x::l) [1;2;3;4;5] [] ;;
- : int list = [1; 2; 3; 4; 5]
```

```
# fold_left_via_fold_right (fun l x -> x::l) [] [1;2;3;4;5] ;;
- : int list = [5; 4; 3; 2; 1]
# fold_right_via_fold_left (fun x y -> 10*y+x) [1;2;3;4;5] 0 ;;
- : int = 54321
# fold_left_via_fold_right (fun x y -> 10*x+y) 0 [1;2;3;4;5] ;;
- : int = 12345
```

考察

まず `fold_right` を `fold_left` で表す方法について考察する。右畳み込み `fold_right f l e` と左畳み込み `fold_left f e l` の対称性と発展 1 でやったように `reverse` が `fold_left` で簡単にかけることに着目する。`f` で `l` を右畳み込みしたものは `l` を `reverse` したものを `f` の引数をひっくり返した関数 (すなわち `fun x y -> (f y x)`) で左畳み込みしたものと同じであり、`reverse` も左畳み込みも `fold_left` のみでかけるので `fold_right` が `fold_left` で書ける。

次に `fold_left` を `fold_right` で書く方を考察する。パズルのように直接的に表そうとしたが難しかったので `fold_right g l [何か]` が、初期値 `e` を受け取って `f, l` での右畳み込みの結果 `fold_left f e l` を返すような関数、を返すように `g(f` に依存) を定めることを考えてみる。`g` がそのようなものであると仮定する。`l=x::xs` とすると `g` が満たすべき式は $(\text{fold_right } g \ l \ \text{[何か]}) \ e = (\text{fold_right } g \ xs \ \text{[何か]}) \ (f \ e \ x)$ である。簡単のために $h = (\text{fold_right } g \ xs \ \text{[何か]})$ と置こう。`g` は `x` と関数 `h(xs` の `g` での右畳み込み結果) を受け取って、関数 $e \mapsto h(f \ e \ x)$ を返す関数であるから $G = \text{fun } f \ x \ h \rightarrow (\text{fun } e \rightarrow h(f \ e \ x)), g = G \ f$ とすればよいことがわかる。残るは [何か] としていた初期値であるが、リストが空のときは初期値をそのまま返せば良いので `id` とすればよい。以上で `fold_left` を `fold_right` で表せた。

発展 3

動作例

```
# eight = add {t=(fun f x -> f (f (f (f (f x)))))}
  {t=(fun f x -> f (f (f x)))};;
val eight : church = {t = <fun>}
# eight.t (fun x -> x+1) 0;;
- : int = 8
# eight.t (fun x -> x+.1.0) 0.0;;
- : float = 8.
# (mul {t=(fun f x -> f (f (f (f (f x))))}
  {t=(fun f x -> f (f (f x)))}.t (fun y -> y+1) 0;;
- : int = 15
# (sub {t=(fun f x -> f (f (f (f (f x))))}
  {t=(fun f x -> f (f (f x)))}.t (fun y -> y+1) 0;;
- : int = 2
```

考察

まず advanced3.ml の実装をしたが型に `_` がついてがおかしくて一度使用したら型が変わっていたり、エラーが出て諦めていたが、第二回の授業スライドでそれが未決定な単相型で、引数に多相関数を渡したいときはレコードを用いるとよいとあったのでそれを用いて advanced3-type-church.ml を書いた。まずは型を考慮せずにどのような方針で add,mul,sub を実装したかについて説明する。

型のことは考えずに

$f^{n+m}(x) = f^n(f^m(x)), f^{nm}(x) = (f^m)^n(x)$ であるから、add,mul はそれぞれ

```
fun n m -> (fun f x -> n f (m f x))  
  
fun n m -> (fun f x -> n (m f) x)
```

のような感じで実装できる。add が実装できるので 1 増やす関数 succ も同様に実装できる。`n` をもらって 1 つ前の $\max(0, n-1)$ を返す関数 pred が実装できれば sub は、let sub = fun n m -> m pred n として実装できるので pred の実装を考える。next_pair (n,m) -> (succ n,n) という関数を考えると $n \text{ next_pair } (\text{zero}, \text{zero}) = (n, n-1)$ なので pred は fun n -> snd (n next_pair ((fun f x -> x), (fun f x -> x))) のように実装できる。

型がどうまずくなったか

上記のように実装するとはじめに述べた通り、add や mul の戻り値は多相関数ではなくて (`'a -> 'a`) -> `'a -> 'a` のように未決定の単相型になってしまうので型が一度決まると変更できなくなる (問題の趣旨は多相関数を返すこと)。

レコードで回避

第二回授業スライドの通り type church = t : 'a. ('a -> 'a) -> 'a -> 'a で定義される church を用いることで引数に多相関数を渡すことができる。この church 型を用いて書き直したのが advanced3-type-church.ml である。ここでは add,mul,sub の型はすべて church -> church -> church である。これで型がついたが、問題の add,mul,sub の要件は関数をもって返す関数であるから入力の際に関数を church 型へ、出力の際に church 型から多相関数へ変換してやる必要がある。

型の変換

これは church 型の引数に対して t をとればいだけだと思っていたが、let ctof = fun c -> c.t として ctof zero などを表示してみると型は (`'a -> 'a`) -> `'a -> 'a` になってしまう。この原因を探してみると、第 2 回の授業スライド内の Value Restriction によるもののように、関数適用は値でない式なので多相的な型が与えられない。実際スライドにあるように (fun f x -> zero.t f x) は多相関数になった。しかし様々な church 型の値を変換する際には関数適用を避ける方法がわからずどのようにしても未決定な単相型の関数を返すようにしかできな

かった。動作例には church 型のままのものが正しく動き、多相関数になっていることを示す例を載せた。